
Práctica 2 - Sensores en el Robot.

Mario Bravo López y Alejandro Otero Vázquez

Fecha de realización: 15 de Abril 2024

En este informe se comentarán los pasos que se siguieron para poder realizar la práctica 2 de la asignatura Robótica y Mecatrónica. Los principales objetivos de esta práctica son el control mediante lazo cerrado del robot, para lo cual es necesario el uso de sensores que nos proporcionen información sobre el entorno. En concreto, en nuestro caso, se optó por el uso del Sharp GP2D12 como sensor de distancia. Se trata de un sensor que funciona con luz de longitud de onda infrarroja y que proporciona en la salida de uno de sus pines una tensión en función de la distancia al objeto medido. De forma paralela se usarán sensores de proximidad CNY70 que ejercerán la función de encoders, con lo que podremos tener un control más preciso sobre la orientación del robot y su velocidad. En los siguientes apartados se hará una explicación más exhaustiva del funcionamiento de estos sensores.

1 Montaje sobre el Robot

Todo buen proyecto en el ámbito de la Robótica, deposita gran parte de su éxito en cuanto bien se realiza el montaje del robot, pues esto nos ahorrará problemas futuros. Una buena implementación del Hardware dará pie a un Software en principio más sencillo.

1.1 Sharp GP2D12

Como ya hemos comentado en esta práctica se usarán dos tipos de sensores distintos. Los sensores para la detección de obstáculos, Sharp GP2D12 se ubicarán en la parte delantera del robot, de manera simétrica respecto al centro del robot, consiguiendo así realizar medidas homogéneas para la distancia frontal. Cabe destacar que para poder realizar el anclaje de estos sensores al cuerpo del robot se usaron tornillos. Como se puede observar en la siguiente imagen:

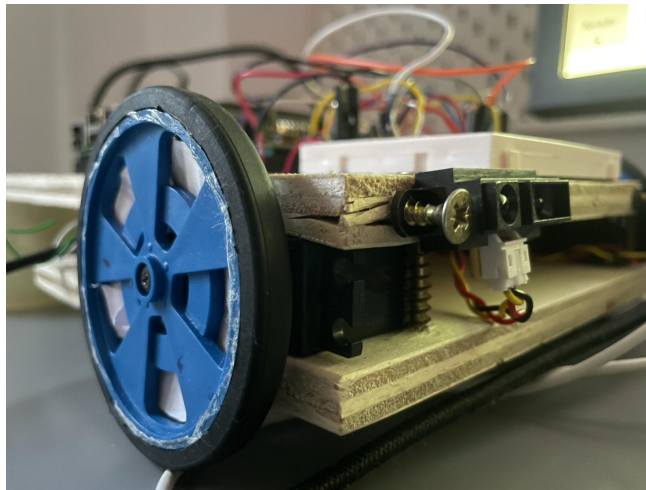


Figure 1: Instalación de los sensores Sharp GP2D12

Este tipo de sensor electro-óptico se ha de alimentar para su correcto funcionamiento con 3.3 V, hay que destacar que esta compuesto por un PSD (position sensitive detector), un IR-LED (infrared emitting diode) y un circuito para procesar la señal. Evidentemente la salida del sensor será de tensión y estará relacionada con la distancia de la siguiente manera:

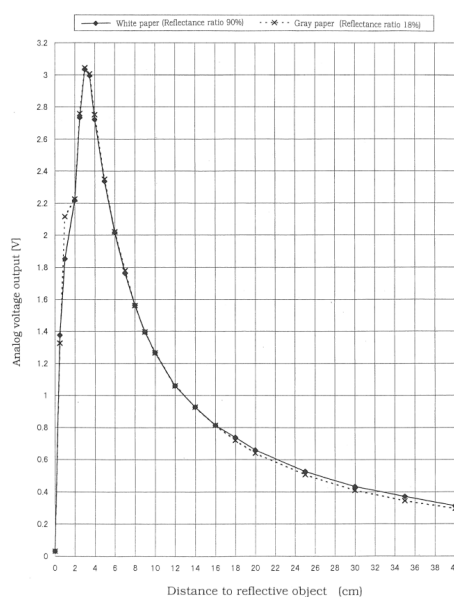


Figure 2: Relación distancia voltaje sensor GP2Y0A41SK0F

Esta relación entre la tensión de la salida y la distancia tiene un problema la multievaluación, pues como se observa en la gráfica adjunta, para distancias diferentes tenemos asociados tensiones iguales. La solución a este problema será considerar que todas las tensiones que se midan serán para distancias a la derecha del máximo presente en la gráfica y consecuentemente como se puede observar en la gráfica las distancias a medir tienen que ser superiores a 3 cm para seguir el criterio que hemos comentado.

Nos gustaría recordar que la salida de este sensor será una tensión por lo tanto deberemos hacer uso del ADC de la Raspberry Pi, además de realizar la configuración de la comunicación SPI usando *mcp3004Setup* (*pinbase*, *spi_channel*). Volviendo al problema de la calibración del sensor Sharp GP2D12, en nuestro caso se ha optado por calibrar el sistema directamente a partir de la gráfica que brinda el datasheet. Obteniendo la relación lineal que existe entre los pixeles de la imagen con los valores para distancia y voltaje somos capaces de muestrear la función con intervalos muy bajos. Al haber tan poco espacio entre valores consecutivos puede aproximarse linealmente la función en cada intervalo y ponderar de este modo cada valor analógico medido por el sensor.

Para obtener la relación entre la imagen y los valores de las variables se ha utilizado esta web: <https://apps.automeris.io/wpd/> siguiendo las indicaciones del siguiente vídeo de youtube: https://www.youtube.com/watch?v=jXTgLhKLhSo&ab_channel=Latex%26Matlab. El estudio da como resultado un archivo csv, de nombre *calibracion.csv* que contiene 70 muestras de valores de distancia y voltaje obtenidos a partir de la gráfica. Este método en principio puede parecer muy eficiente y correcto, pues a fin de cuentas estamos ciñéndonos al datasheet del sensor y fue el que en un principio se iba a usar.

Sin embargo, hemos de tener en cuenta que estos valores son los que se tendrían de un sensor perfecto, nuevo y bajo unas condiciones de iluminación externas que se desconocen. Lo cual provoca que si se usa la relación de datos distancia-tensión de este archivo las medidas de distancia, realizadas por el sensor no se acercan nada a las reales. Es por ello que tras pensar en posibles soluciones al desajuste del sensor a los valores teóricos del datasheet. Se optó por realizar un programa que calibrará los sensores, *Calibracion_distancia.c*, consiguiendo generar así un archivo con valores para la relación, distancia-tensión de los sensores que se ajuste mejor a las condiciones de iluminación.

1.2 CNY70

El siguiente frente que se nos presenta a la hora de realizar el montaje para esta práctica es la introducción de encoders en el robot, los cuales tienen que encontrarse muy cerca de las ruedas, a las cuales se les deberá añadir un patrón de franjas negras y blancas, en nuestro caso se optó por usar el siguiente software proporcionado en la asignatura: <https://code.google.com/archive/p/wheel-encoder-generator/>, para diseñar un patrón de 20 franjas blancas y negras.

La razón por la cual se usan franjas negras y blancas viene dictado por el propio funcionamiento de los sensores CNY70, pues si hacemos caso al datasheet de los mismos, podremos comprender que la tensión de salida que proporciona este sensor de distancia nos depende de la reflectividad de la superficie sobre la que incide el haz infrarrojo. El sensor CNY70 está formado por un emisor de infrarrojo de $\lambda = 950 \text{ nm}$ y un foto transistor, que será el encargado de dar una señal de tensión en función de la radiación que detecte.

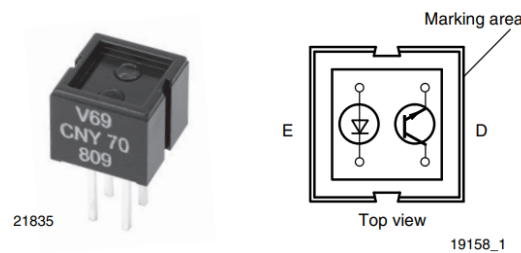


Figure 3: Caption

Es por ello que la superficie sobre la que se refleje el haz infrarrojo va a tener un gran impacto sobre la tensión de salida del sensor. En concreto en el caso de las franjas negras y blancas, podemos observar como, para las franjas negras estas absorberán una mayor parte de la radiación emitida y consecuentemente la radiación restante que llegue al foto transistor será menor, con lo que la salida en tensión disminuirá. En el caso de las franjas blancas de manera inversa podemos observar un aumento en la señal de salida. Consecuentemente para conseguir un correcto funcionamiento de nuestro software es necesario calibrar con bastante precisión que tensión tiene asociada el color blanco y el color negro, pues como se comenta en el apartado de **Software de control** el estado del encoder dependerá de las tensiones umbrales que se hayan definido para cada encoder en específico. En nuestro caso tras conseguir montar de manera óptima los encoders, se comenzó a medir con el ADC las salidas de tensión que nos proporcionaban los diversos sensores CNY70, consiguiendo establecer la siguiente relación:

	Encoder Izquierdo	Encoder Derecho
V_{negro}	$< 12 V_{LSB}$	$< 200 V_{LSB}$
V_{blanco}	$> 25 V_{LSB}$	$> 400 V_{LSB}$

Estos niveles se han definido con el objetivo de evitar errores en la detección de franjas.

2 Software de control.

Desde una vista de alto nivel, el código está estructurado en 5 hilos. Cada hilo ejecuta una función paralela al resto de hilos de tal forma que las tareas que debe realizar cada elemento del robot se paraleliza con las demás. Cada hilo estará asociado generalmente con un elemento del hardware (sensor).

Cada hilo se inicia con una función específica (distanceMeasureRight, distanceMeasureLeft, encoderLeft, encoderRight, movement) y comparten una estructura de datos ThreadArgs o sharedData.

2.1 Definición de estructura de datos.

Durante todo el programa las funciones trabajarán con punteros a un tipo de estructura definida como ThreadArgs. Esta estructura contiene todas las variables que necesitarán compartir los hilos entre sí para el funcionamiento del robot. Por ejemplo, los punteros *voltageLft y *voltageRht apuntan a una dirección de memoria donde se almacenará un array de datos con los voltajes que presentan los sensores Sharp a distintas distancias. Estas distancias se almacenarán en la dirección de memoria a la que apunta *x. Trabajar con punteros es muy útil al utilizar arrays. Permite acceder a la dirección de memoria donde están guardados esos datos sin necesidad de cargarlos en un nuevo array cada vez que se vayan a necesitar. Por otra parte se definen 4 variables con las distancias que medirán los dos sensores (distLeft, distRight) y el número de cuentas que lleve cada rueda (counterLeft, counterRight). Estas 4 variables serán las que evaluarán los hilos asociados a cada sensor modificando su valor. Por último se definen 4 mutex asociados a estas 4 variables para bloquear su acceso al resto de hilos cuando uno de ellos esté accediendo a ellas. La variable auxiliar salir se utiliza para forzar a los hilos a salir de su ejecución cuando el movimiento del robot termine y que así finalice el programa. pthread_barrier_t se utiliza para sincronizar múltiples hilos en un punto específico de ejecución. Proporciona un mecanismo para que los hilos esperen unos a otros hasta que todos alcancen cierto punto de ejecución antes de continuar.

```
typedef struct {
    float distLeft;
    float distRight;
    int counterLeft;
    int counterRight;
    pthread_mutex_t mutexLeft;
    pthread_mutex_t mutexRight;
    pthread_mutex_t mutexEncoderLeft;
    pthread_mutex_t mutexEncoderRight;
    float *x;
    float *voltageLft;
    float *voltageRht;
    pthread_barrier_t barrera;
    int salir;
} ThreadArgs;

typedef struct {
    float *x_cal;
    float *voltageLft_cal;
    float *voltageRht_cal;
    int longitud;
} DatosArchivo;
```

2.1.1 Inicio y destrucción de la estructura.

Función initSharedData:

```
ThreadArgs* initSharedData() {
    ThreadArgs *data = malloc(sizeof(ThreadArgs));
    if (data == NULL) {
        fprintf(stderr, "Error de asignación de memoria\n");
        exit(EXIT_FAILURE);
    }
    const char *nombreArchivo = "cal_datos_sharp.txt";

    DatosArchivo medidas = leerDesdeArchivo(nombreArchivo);
```

```

data->x = medidas.x_cal;
data->voltageLft = medidas.voltageLft_cal;
data->voltageRht = medidas.voltageRht_cal;

data->distLeft = 0.0;
data->distRight = 0.0;
data->counterLeft = 0;
data->counterRight = 0;
data->salir = 0;
pthread_mutex_init(&data->mutexLeft, NULL);
pthread_mutex_init(&data->mutexRight, NULL);
pthread_mutex_init(&data->mutexEncoderLeft, NULL);
pthread_mutex_init(&data->mutexEncoderRight, NULL);
pthread_barrier_init(&data->barrera, NULL, 3);
return data;
}

```

Esta función inicializa una estructura ThreadArgs y reserva memoria dinámicamente para ella utilizando malloc. Si la asignación de memoria falla, se emite un mensaje de error y el programa se cierra. Posteriormente se usa la función leerDesdeArchivo() y la estructura DatosArchivos, para cargar los valores de la calibración de tensión y distancia de los sensores Sharp realizada por el programa Calibracion_distancia.c, cuyo funcionamiento se explica al final de la sección. A continuación, se inicializan todos los campos de la estructura ThreadArgs, estableciendo las variables distLeft, distRight, counterLeft, counterRight y salir en cero. Además, se inicializan los mutex mutexLeft, mutexRight, mutexEncoderLeft y mutexEncoderRight utilizando pthread_mutex_init, y se inicializa una barrera barrera con un recuento de 3 hilos utilizando pthread_barrier_init. Finalmente, la función devuelve un puntero a la estructura inicializada.

Función destroySharedData:

```

void destroySharedData(ThreadArgs *data) {
    pthread_mutex_destroy(&data->mutexLeft);
    pthread_mutex_destroy(&data->mutexRight);
    pthread_mutex_destroy(&data->mutexEncoderLeft);
    pthread_mutex_destroy(&data->mutexEncoderRight);
    pthread_barrier_destroy(&data->barrera);
    free(data->x);
    free(data->voltageLft);
    free(data->voltageRht);
    free(data);
}

```

Esta función libera la memoria y destruye los objetos de sincronización asociados con la estructura ThreadArgs. Primero, destruye los mutex mutexLeft, mutexRight, mutexEncoderLeft y mutexEncoderRight utilizando pthread_mutex_destroy. Luego, destruye la barrera barrera utilizando pthread_barrier_destroy. Después, libera la memoria reservada para los punteros x, voltageLft y voltageRht utilizando free. Finalmente, libera la memoria de la propia estructura ThreadArgs utilizando free. Esta función debe ser llamada al finalizar el uso de la estructura para evitar pérdidas de memoria y liberar los recursos de sincronización.

2.2 ¿Por qué trabajar con punteros a estructuras?

En líneas generales las funciones que implementa el código trabajan con argumentos puntero a estructuras ThreadArgs en lugar de trabajar con variables locales. Esto se debe a que trabajar con una estructura ThreadArgs requiere cargar sobre las direcciones de memoria los valores de los arrays de distancia y voltaje de los sensores Sharp, calibrados previamente, cada vez que se quiera definir un nuevo ThreadArgs. Además, las funciones de los hilos al trabajar con punteros serán capaces de modificar el valor almacenado en la

dirección de memoria de la variable a la que accedan. De esta forma, cuando otra variable acceda a esa dirección de memoria el valor estará actualizado y las funciones podrán estar conectadas.

2.3 Función `main()`.

- **Inicialización del hardware:**

- Se inicializa el sistema de pines WiringPi mediante `wiringPiSetup()`.
- Se configuran dos pines como salidas PWM (modulación de ancho de pulso) mediante `pinMode()`.
- Se establece el modo PWM mediante `pwmSetMode()`.
- Se establece el rango de PWM mediante `pwmSetRange()`.
- Se establece el divisor de reloj PWM mediante `pwmSetClock()`.

- **Configuración de la comunicación SPI:**

- Se llama a `wiringPiSPISetup()` para configurar la comunicación SPI con un cierto canal y velocidad.
- Se utiliza `mcp3004Setup()` para configurar un convertidor analógico a digital (ADC) MCP3004.

- **Creación de hilos:**

- Se crean cinco hilos diferentes (`thread1`, `thread2`, `thread3`, `thread4`, `thread5`) utilizando `pthread_create()`.
- Cada hilo se inicia con una función específica (`distanceMeasureRight`, `distanceMeasureLeft`, `encoderLeft`, `encoderRight`, `movement`) y comparten una estructura de datos `ThreadArgs` o `sharedData`.
- Si la creación del hilo falla, se imprime un mensaje de error.

- **Espera a que los hilos terminen:**

- Después de crear todos los hilos, el programa principal espera a que cada uno de ellos termine utilizando `pthread_join()`.

- **Impresión de un mensaje:**

- Finalmente, el programa principal imprime un mensaje que indica que ha terminado la ejecución de todos los hilos.

Es importante tener en cuenta que la estructura `ThreadArgs` o `sharedData` es compartida entre los hilos para pasar información entre ellos de manera segura. Para ello se utilizan mecanismos de sincronización mutex para garantizar el acceso seguro a los datos compartidos.

2.4 Funciones de los hilos.

2.4.1 `void *distanceMeasureRigth(void *args_ptr)`

Esta función se encarga de medir la distancia desde un sensor infrarrojo ubicado en el lado derecho de un sistema, y actualizar continuamente la distancia medida en una estructura de datos compartida. Aquí está el desglose de lo que hace:

1. **Inicialización y sincronización:**

- La función espera en una barrera hasta que la calibración de los sensores esté lista. Al comienzo de su ejecución lee los valores almacenados en los arrays de voltaje y posición, que como ya se ha comentado obtienen los valores del archivo de calibración `cal_datos_sharp.txt`. Si la calibración no ha sido completada antes de que cargue los datos no será capaz de medir. Es por esto que se utiliza una barrera hasta que finaliza la calibración.

2. Obtención de datos de medición:

- Lee la posición actual del objeto mediante un sensor analógico y la convierte en voltaje.
- Aplica un filtro de media móvil ponderada para suavizar las lecturas de voltaje y reducir el ruido.
- Calcula la distancia a partir del voltaje medido utilizando una función `getDistance()` que toma en cuenta la relación entre la distancia y el voltaje según la configuración del sensor.

3. Actualización de datos compartidos:

- Bloquea el acceso a la variable `distRight` mediante un mutex para garantizar que ningún otro hilo modifique los datos mientras se están actualizando.
- Actualiza la distancia medida en la estructura de datos compartida `args->distRight`.
- Desbloquea el mutex para permitir que otros hilos accedan a los datos.

4. Control de tiempo:

- Ajusta el tiempo de muestreo para asegurar una medición regular, esperando el tiempo restante si la medición tomó menos tiempo de lo esperado.

5. Terminación del hilo:

- Verifica si se ha establecido la bandera `args->salir`, que indica si el hilo debe finalizar.
- Si se establece la bandera, el hilo sale de la función utilizando `pthread_exit()`.

En resumen, esta función encapsula el proceso de medición de distancia desde un sensor de ultrasonido en el lado derecho del robot, actualizando continuamente la distancia medida en una estructura de datos compartida, y asegurando que la medición se realice de manera sincronizada y controlada.

2.4.2 void *distanceMeasureLeft(void *args_ptr)

El funcionamiento de `distanceMeasureLeft` es similar al de `distanceMeasureRight`, con la diferencia de que esta función accede a la dirección de memoria de `voltageLft` y actualiza la variable `args->distLeft`.

2.4.3 void *encoderLeft(void *args_ptr)

Función `encoderLeft`:

Esta función se encarga de monitorear el encoder de la rueda izquierda del robot y contar los pulsos generados por el movimiento del encode. Aquí está el desglose de lo que hace:

1. Inicialización y obtención de datos compartidos:

- La función recibe un puntero a una estructura de datos compartida `ThreadArgs`, que contiene información necesaria para la ejecución del hilo.

2. Configuración de parámetros:

- Se establece el tiempo de muestreo para determinar con qué frecuencia se leerá el estado del encoder.

3. Bucle de medición:

- La función entra en un bucle infinito que monitorea continuamente el estado del encoder.
- Lee el estado actual del encoder (alto o bajo) utilizando la función `analogRead()` y lo almacena en el array `measure`.
- Se establecen unos valores limitantes del retorno de `analogRead()` a partir de los que se establece que el sistema está en alta o baja. Estos valores de han determinado experimentalmente y dependen del sensor que se utilice.

- Si se detecta un cambio de estado entre las lecturas actual y anterior, lo cual indica un pulso del encoder, se incrementa el contador de pulsos `counterLeft` y se actualiza en la estructura compartida `sharedData`.
- Se imprime el valor del contador `counterLeft` para su visualización en la salida estándar.

4. Control de tiempo:

- Se calcula el tiempo transcurrido entre cada iteración del bucle y se espera el tiempo restante del ciclo de muestreo si el bucle se ha ejecutado más rápido de lo esperado.

5. Terminación del hilo:

- Verifica si se ha establecido la bandera `sharedData->salir`, que indica si el hilo debe finalizar.
- Si se establece la bandera, el hilo sale de la función utilizando `pthread_exit()`.

En resumen, esta función controla el contador de pulsos de un encoder rotativo, actualizando continuamente el conteo de pulsos en una estructura de datos compartida y asegurando que la medición se realice de manera sincronizada y controlada.

2.4.4 void *encoderRigth(void *args_ptr)

Esta función se encarga de monitorear el encoder de la rueda derecha del robot y contar los pulsos generados por el movimiento del encode. Su funcionamiento es exactamente el mismo que el de `void *encoderLeft(void *args_ptr)` excepto que esta función actualiza la variable `args->counterRight`.

2.4.5 void *movement(void *args_ptr)

La función `movement` gobierna totalmente el movimiento del robot. Se han definido funciones auxiliares que simplifican la sintaxis de este hilo. Sin entrar en detalle de momento sobre como funcionan, este hilo básicamente es configurable a la trayectoria que le quieras dar al robot. Para su movimiento hace uso de las funciones: `motorMoveRelative()` y `MotorRotateAngle()` que moverán el robot en línea recta y lo girarán. Antes de comenzar a mover el robot se utiliza una barrera de sincronización (`pthread_barrier_wait()`) con el objetivo de asegurarnos que se han cargado los valores de la calibración y también para sincronizar con hilos de medida para que empiecen a medir.

2.5 Funciones auxiliares

2.5.1 DatosArchivo leerDesdeArchivo(const char *nombreArchivo)

Esta función inicializa una estructura `DatosArchivo`, donde se guardaran los datos cargados del archivo de calibración correspondientes a tensión de los sensores y distancia.

1. Obtención del número de filas del archivo de calibración, pues en principio se pueden desconocer el número de puntos que se han tomado para calibrar los sensores Sharp.
2. Asignación de memoria dinámica, conocidas el número de filas totales del archivo se usa la función `malloc()` para reservar memoria dinámica.
3. Por último se leen los datos del archivo usando `fscanf()` y se asocian a las variables de la estructura `datos x_cal`, `voltageLft_cal` y `voltageRht_cal`. Que será lo que la función nos devuelva al llamarla en el código.

2.5.2 void motorMoveRelative(int security, float mm, void *args_ptr)

Esta función implementa el movimiento del robot una distancia de `mm` milímetros teniendo en cuenta una distancia de seguridad `security`. Para ello debe acceder a las variables compartidas de distancia y cuentas de los sensores.

1. Calcula el número de cuentas necesarias para moverse la distancia solicitada.
2. Mientras que no se alcance el número de cuentas esperado por uno de los encoders o se haya detectado un objeto por alguno de los sensores a una distancia menor de la de seguridad el robot se moverá e irá actualizando las variables de distancia y contadores que vayan midiendo el resto de hilos. Si una de las ruedas recorre más de dos cuentas que la otra se modifican las velocidades de los motores para que el robot se reoriente.
3. En caso de que el robot se haya parado porque un objeto ha sido detectado y no porque haya llegado a su destino se acercará a una distancia menor aminorando la velocidad. Cuando se haya acercado el usuario dispondrá de 10 segundos para retirar el objeto antes de que se termine la ejecución del programa.
4. En caso de que el objeto se haya retirado a tiempo se llama de nuevo a la misma función para que repita de nuevo el proceso. La variable mm no se modifica porque el sistema no ha reseteado el número de cuentas y conoce la distancia que lleva recorrida.

2.5.3 void motorRotateAngle(int angle, void *args_ptr)

Esta función implementa el giro del robot un arco de angle grados. Para ello debe acceder a las variables compartidas de cuentas de los encoders.

1. Calcula el número de cuentas necesarias para girar el arco solicitado.
2. Dependiendo del signo del ángulo se le da una velocidad distinta a los motores para que giren en un sentido o en otro.
3. Mientras que no se alcance el número de cuentas esperado por los encoders el robot se moverá e irá actualizando las variables de contadores que vayan midiendo el resto de hilos.
4. Una vez que se hayan alcanzado el número de cuentas objetivo el robot se para.

Esta función no tiene en cuenta los sensores de distancia porque gira sobre el robot gira sobre sí mismo y no hay peligro de impacto.

2.5.4 float getDistance(float measure, float *x, float *voltage)

Este código implementa una función llamada getDistance que calcula la distancia correspondiente a una medida analógica dada utilizando una interpolación lineal. Aquí está cómo funciona:

1. Parámetros de entrada
 - measure: La medida analógica obtenida del sensor, que se desea convertir en una distancia.
 - x: Un puntero a un array que contiene las distancias correspondientes a las medidas de voltaje.
 - voltage: Un puntero a un array que contiene las medidas de voltaje correspondientes a las distancias en el array x.
2. Inicialización:
 - Se declara una variable dist para almacenar la distancia calculada.
 - Se inicializa un contador n en cero para recorrer los arrays x y voltage.
3. Bucle de Búsqueda:
 - Se recorre un bucle while mientras n sea menor que la dimensión de los arrays x y voltage.

- En cada iteración del bucle, se compara la medida `measure` con el valor de voltaje actual (`*voltage`). Si `measure` es menor que el valor de voltaje actual, se incrementan los punteros `x` y `voltage` para pasar al siguiente par de valores.
- Se incrementa el contador `n` para pasar al siguiente índice en los arrays.

4. Cálculo de Fracción:

- Después de salir del bucle, se calcula el intervalo entre los dos voltajes más cercanos (`interval`) como la diferencia entre el voltaje actual y el voltaje anterior en el array `voltage`.
- Se calcula una fracción (`frac`) que representa la posición relativa de `measure` dentro de este intervalo. Esto se hace restando el voltaje anterior de `measure` y dividiendo por el intervalo.

5. Interpolación Lineal:

- Finalmente, se utiliza la interpolación lineal para calcular la distancia correspondiente a la medida `measure`. Esto se hace tomando la distancia correspondiente al voltaje anterior (`*(x-1)`) y sumando la fracción del intervalo multiplicada por la diferencia entre las distancias en los arrays `x`.

6. Retorno de Resultado:

- Se devuelve la distancia calculada.

En resumen, esta función toma una medida analógica de un sensor y utiliza una interpolación lineal para calcular la distancia correspondiente basada en los datos de calibración proporcionados en los arrays `x` y `voltage`.

2.6 Software de calibración

Como se ha comentado a lo largo del documento para poder usar de manera correcta los sensores Sharp, bajo distintos tipos de iluminación la solución más sensata que se nos ocurrió fue realizar un programa que creará un archivo con los valores de distancia y tensión medida por el ADC para cada sensor y así poder medir con mayor precisión la distancia del robot a los objetos de su entorno. El programa que realiza este proceso, `calibracion_distancia.c`, debe ejecutarse previo a usar el robot en un ambiente nuevo de iluminación.

De manera muy simple el programa tiene una estructura muy similar al propio programa de funcionamiento del robot, solo que usa tres hilos, consiguiendo realizar múltiples tareas complejas de manera paralela. Al igual que en el software de ejecución del robot se usan dos hilos para el conteo de franjas y un tercer hilo que llevará asociada la función `void *calDistance(void *data_ptr)`, para poder realizar la calibración de manera óptima se recomienda acercar el robot a 5 cm de una superficie de color preferiblemente blanca y con los sensores Sharp orientados en tal dirección, el robot de manera automática irá variando su distancia respecto a esta superficie y almacenará la tensión que proporcionen los sensores de distancia.

2.6.1 Generación y destrucción de estructuras

En este programa también se hará uso de estructuras referenciadas con puntero, para poder compartir múltiples variables entre distintas funciones del código. Para ello se usó una estructura similar a la ya presentada previamente, pero con menos variables pues se trabaja con menos hilos:

```
typedef struct {
    int counterLeft;
    int counterRight;
    pthread_mutex_t mutexEncoderLeft;
    pthread_mutex_t mutexEncoderRight;
```

```

float *x;
float *voltageLft;
float *voltageRht;
pthread_barrier_t barrera;
int salir;
} ThreadArgs;

```

En esta estructura además de los ya explicados contadores de las ruedas, que nos servirán para conocer el estado del robot, junto a sus respectivas variables de tipo mutex, para poder establecer un orden a la hora de cargar los valores de estas dos variables. Se crean tres variables de tipo puntero, en las cuales se almacenarán la distancia recorrida por el robot y la tensión de los sensores Sharp con el objetivo de posteriormente generar el archivo de calibración.

2.6.2 Inicialización y destrucción de memoria dinámica

Como ya se ha comentado este segundo código, para realizar la calibración, tiene muchas partes en común con el código de control del robot. Como son la inicialización de las variables y su posterior destrucción, estas funciones se encuentran como es de esperar dentro de `main()`:

1. `ThreadArgs* initSharedData()`: Reserva memoria dinámica usando `malloc()` e inicializa a cero los contadores de las ruedas.
2. `void destroySharedData(ThreadArgs *data)` Libera la memoria dinámica reservada para la estructura y además elimina los mutex creados para la sincronización de variables.

2.6.3 Funciones de hilos

1. `void *encoderLeft(void *args_ptr)` y `void *encoderRigth(void *args_ptr)`: Ambas funciones se encargan de dar información del encoder de su respectiva rueda, midiendo los pulsos generados al rotar la rueda. Siguen una lógica igual que la ya explicada previamente, en el código de control del robot, por lo que si se tiene dudas respecto al funcionamiento de las mismas, se recomienda volver a leer dicho apartado.
2. `void *calDistance(void *data_ptr)`: Esta función será ejecutada por el tercer hilo y será la responsable de coordinar el movimiento del robot junto a la adquisición de datos mediante el ADC de los sensores de distancia Sharp.

Se le pregunta al usuario la precisión en cm que necesita para su calibración. Y posteriormente se calcula el número de medidas necesarias, tomando como distancia total que recorrerá el robot 30 cm. Esta elección se hizo en base a la inestabilidad del sensor para distancias superiores. Para cada distancia en la que se debe detener el robot para medir, se calcula su equivalente en franjas y se realizan bucles de tipo `dowhile()` en función de las franjas que haya contado cada encoder, si por algún casual el robot se desvía de su trayectoria teóricamente recta, se varia la velocidad de los servos para poder reajustar la trayectoria del robot.

Una vez alcanzada la distancia donde se quiere realizar una medida del voltaje que nos entregan los sensores Sharp, el robot se para y durante 5 segundos se realiza una media de los valores de tensión que da el sensor. Tanto el valor medio como la distancia a la que se realiza la medida se almacenan en vectores, que serán las variables de entrada de una función auxiliar para crear un archivo de texto con estas medidas.

2.6.4 Función auxiliar:

```
void guardarEnArchivo(const char *nombreArchivo, float array1[], float array2[],float  
array3[], int longitud)
```

Esta función toma como parámetros de entrada el nombre del archivo en el cual se quieren guardar los arrays de enteros, siendo array1[] el asociado a las distancias, array2[] el de tensiones del sensor izquierdo y array3[] el de tensiones del sensor derecho. El código no tiene complejidad alguna, conocidas las dimensiones de los vectores se recorren elemento a elemento y usando la función `fprintf()` se va almacenando en columnas separadas separadas por comas los valores de distancia y tensiones.